## Day- 2  Data Structures in Python

## 1.Python String:

Strings are sequence of characters written inside quotes. It can include letters, numbers, symbols and spaces. Python does not have a separate character type.

- A single character is treated as a string of length one.
- Strings are commonly used for text handling and manipulation.

a = 'Jyoti'

b = "Verma"

print(a)

print(b)

**Multi-line Strings:**

Use triple quotes ('''...''' ) or ( """...""") for strings that span multiple lines. Newlines are preserved.

s = """I am Learning

Python String """

print(s)

s = '''I'm a Student'''

print(s)

**String Slicing:**

Slicing is a way to extract a portion of a string by specifying the start and end indexes. The syntax for slicing is string[start:end], where start starting index and end is stopping index (excluded).

**Example:**

s = "ABCDEF"

print(s[1:4])

print(s[:3])

print(s[3:])

print(s[::-1])

**Common String Methods:**

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. **len():** returns the total number of characters in a string (including spaces and punctuation).
2. **upper() and lower():** upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.
3. **strip() and replace():** strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

**Example:**

```
s = "   Hello World! Welcome to Python.   "

print("Original String:", s)

print("Length:", len(s))

print("Uppercase:", s.upper())

print("Lowercase:", s.lower())

print("After strip():", s.strip())

print("After replace():", s.replace("Python", "Java"))
```

**Output:**

```
Original String:    Hello World! Welcome to Python.

Length: 37

Uppercase:    HELLO WORLD! WELCOME TO PYTHON.

Lowercase:    hello world! welcome to python.

After strip(): Hello World! Welcome to Python.

After replace():    Hello World! Welcome to Java.
```

## 2.Python Lists

List is a built-in data structure used to store an ordered collection of items. They are dynamic, resizable and capable of storing multiple data types.

- Mutable: list elements can be changed, updated, added, or removed after the list is created.
- Ordered: elements maintain the order in which they are inserted.
- Index-based: elements are accessed using their position, starting from index 0.

**Example:**

```
a = [1, 2, 3]
```

```
print(a)
b = ["apple", "banana"]
print(b)
```

**Characteristics of Lists:**

1. **Ordered**
    - Elements are stored in a fixed order.
    - Accessed using indexing.
2. **Mutable**
    - Elements can be added, removed, or modified after the list is created.
3. **Allows Duplicate Values**
    - A list can contain the same value multiple times.
4. **Heterogeneous**
    - Can store different data types in the same list.
5. **Dynamic**
    - The size of a list can increase or decrease during program execution.

## Operations on Lists:

1)**Creating a List**

**Example:**

```
numbers = [10, 20, 30, 40]
print(numbers)
```

2) **Accessing Elements:**Using positive indexing

**Example:**

```
numbers = [10, 20, 30, 40]
print(numbers[0])
print(numbers[2])
```

3) **Slicing a List**

**Example:**

```
numbers = [10,20,30,40,50]
print(numbers[1:4])
print(numbers[:3])
```

```
print(numbers[2:])
print(numbers[::-1])
```

**4) Updating (Modifying) Elements**

**Example:**

```
numbers = [10,20,30]
numbers[1] = 25
print(numbers)
```

5) **Adding Elements**

**(i)append():**Adds an element at the end.

**Example:**

```
numbers = [10,20]
numbers.append(30)
print(numbers)
```

**(ii)insert():**Adds an element at a specified position.

**Example:**

```
numbers.insert(1,15)
print(numbers)
```

**(iii) extend():**Adds another list.

**Example:**

```
numbers.extend([40,50])
print(numbers)
```

**6. Deleting Elements**

**(i)remove():**Removes a specified value.

```
numbers.remove(20)
```

**(ii)pop():**Removes an element using its index.

```
numbers.pop(2)
```

**(iii)del:**Deletes an element or the entire list.

```
del numbers[1]
```

**(iv)clear():**Removes all elements.

```
numbers.clear()
```

**Advantages of Lists:**

- Stores multiple values in a single variable.
- Supports indexing and slicing.
- Mutable (easy to modify).
- Can store different data types.
- Provides many built-in methods.

**Disadvantages of Lists:**

- Uses more memory than tuples.
- Searching large lists may be slower.
- Mutable, so data can be modified accidentally**.**

## 3.Python Tuple:

A tuple is a built-in data structure in Python used to store multiple elements in a single variable. Tuples are ordered, immutable (cannot be changed after creation), allow duplicate values, and can store different data types.

**Example:**

fruits = ("Apple", "Banana", "Mango")

numbers = (10, 20, 30, 40)

mixed = (10, "Python", 3.14, True)

**Characteristics of Tuples**

1. Ordered
    - Elements are stored in a fixed order.
    - Each element has an index starting from 0.
2. Immutable
    - Elements cannot be added, removed, or modified after the tuple is created.
3. Allows Duplicate Values
    - A tuple can contain duplicate elements.
4. Heterogeneous
    - A tuple can store different data types.

5. Faster than Lists
    - Tuples are generally faster than lists because they are immutable.

**What is Immutable in Tuples?**

Unlike Python lists, tuples are immutable. Some Characteristics of Tuples in Python.

- Like Lists, tuples are ordered and we can access their elements using their index values
- We cannot update items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.
- Like List Traversal, we can traverse through a tuple using for loop.

  **Example:**

```
# Define a tuple
tup = (1, 2, 3, 4, 5)
# Traverse through each item in the tuple
for x in tup:
    print(x, end=" ")
```

**Built-in Functions Used with Tuples**

| Function | Description |
|---|---|
| len(tuple) | Returns the number of elements |
| max(tuple) | Returns the largest element |
| min(tuple) | Returns the smallest element |
| sum(tuple) | Returns the sum of numeric elements |
| sorted(tuple) | Returns a sorted list from the tuple |

**Example:**

```
t = (10, 40, 20, 30)
print(len(t))
print(max(t))
print(min(t))
print(sum(t))
print(sorted(t))
```

**Advantages of Tuples:**

- Faster than lists.
- Immutable, so data remains safe.
- Uses less memory than lists.
- Can be used as dictionary keys (if all elements are immutable).

**Disadvantages of Tuples:**

- Cannot add, remove, or modify elements.
- Less flexible than lists.

## 4.Python Dictionary

Dictionary is a data structure that stores information in key-value pairs. While keys must be unique and immutable (like strings or numbers), values can be of any data type, whether mutable or immutable. This makes dictionaries ideal for accessing data by a specific name rather than a numeric position like in list.

**Creating a Dictionary**

A dictionary is created by writing key-value pairs inside { }, where each key is connected to a value using colon (:). A dictionary can also be created using dict() function.

**Example:**

```
a = {"x": 1, "y": 2}

print(a)

b = dict(name="Sam", age=20)

print(b)
```

**Characteristics of Dictionary:**

1. Stores data as key-value pairs.
2. Mutable – Values can be added, modified, or deleted.
3. Ordered (Python 3.7 and later).
4. Keys must be unique.
5. Values can be duplicated.
6. Can store different data types.

**1.Accessing Elements:**

Using the key:

```
student = {
    "Name": "Jyoti",
    "Age": 20
}
print(student["Name"])
print(student["Age"])
```

**2. Adding Elements**

```
student = {
    "Name": "Jyoti"
}
student["Branch"] = "CSE"
print(student)
```

**3. Updating Elements**

```
student = {
    "Name": "Jyoti",
    "Age": 20
}
student["Age"] = 21
print(student)
```

**4.Deleting Elements**

**Using pop()**

```
student = {
    "Name": "Jyoti",
    "Age": 20
}
```

```
student.pop("Age")

print(student)
```

**5.Searching Operations**

Check whether a key exists:

```
student = {
    "Name": "Jyoti",
    "Age": 20
}

print("Name" in student)

print("Branch" in student)
```

**6. Built-in Functions Used with Dictionaries:**

| Function | Description |
|---|---|
| len(dict) | Returns the number of key-value pairs |
| max(dict) | Returns the largest key |
| min(dict) | Returns the smallest key |
| sorted(dict) | Returns a sorted list of keys |

Example:

```
student = {
    "B": 80,
    "A": 90,
    "C": 70
}

print(len(student))

print(max(student))

print(min(student))

print(sorted(student))
```

**Advantages of Dictionary:**

- Fast access using keys.
- Stores data in key-value pairs.
- Mutable and easy to update.
- Can store different data types.
- Efficient for searching and retrieving data.

**Disadvantages of Dictionary:**

- Duplicate keys are not allowed.
- Uses more memory than lists or tuples.
- Elements are accessed using keys, not indexes.

## 5.Python Sets

Set is a built-in Python data type used to store a collection of unique items.

- Stores only unique elements; duplicate values are automatically removed.
- Unordered collection, so elements do not have a fixed position and cannot be accessed using indexes.
- Supports fast search, insertion and deletion operations using hashing internally.

**Example:**

```
s = {10, 50, 20}

print(s)

print(type(s))
```

**Methods for Sets:**

**1. Adding elements:** add() function is used to insert new elements into a set. It automatically ignores duplicates.

```
s = {"a", "b", "c"}

s.add("d")

print(s)
```

**2. Union:** union() function combines two sets and returns a new set with all unique elements.

```
a = {"x", "y"}

b = {"y", "z"}
```

```
u = a.union(b)

print(u)
```

**3. Intersection:** intersection() function returns a new set containing elements that are common to both sets.

```
a = {1, 2, 3}

b = {2, 3, 4}

i = a.intersection(b)

print(i)
```

**4. Difference:** difference() function returns a set containing elements that are in the first set but not in the second.

```
a = {1, 2, 3}

b = {2, 3, 4}

d = a.difference(b)

print(d)
```

**5. Clear:** clear() function removes all elements from a set, leaving it empty.

```
s = {1, 2, 3}

s.clear()

print(s)
```